计算机操作系统

第7章 设备管理

设备

处理器和主存储器构成计算机系统硬件的主要部分，其他部分统称为“设备”。

- 硬件层面上，设备由芯片、电路、电源、发动机等物理部件组成。
- 软件层面上，设备是一组指令、状态标志、IO传输过程、错误处理 等。

操作系统关注如何管理和控制设备，而不关心其硬件层面。

设备的分类

设备按用途分为：（外）存储设备和输入/输出设备。

设备按数据传输速度分为：

- 慢速设备——键盘、打印机、鼠标 等
- 快速设备——磁盘、光盘 等

设备按数据组织方式分为：

- 字符设备——以字符为基本单位组织数据（键盘、显示器 等）
- 块设备——以块为基本单位组织数据（磁盘 等）

设备的分类

设备按管理方式分为：

- 物理设备——计算机系统中实际存在的设备实体（设备物理号）
- 逻辑设备——操作系统提供给用户/程序使用的设备（名称/变量）

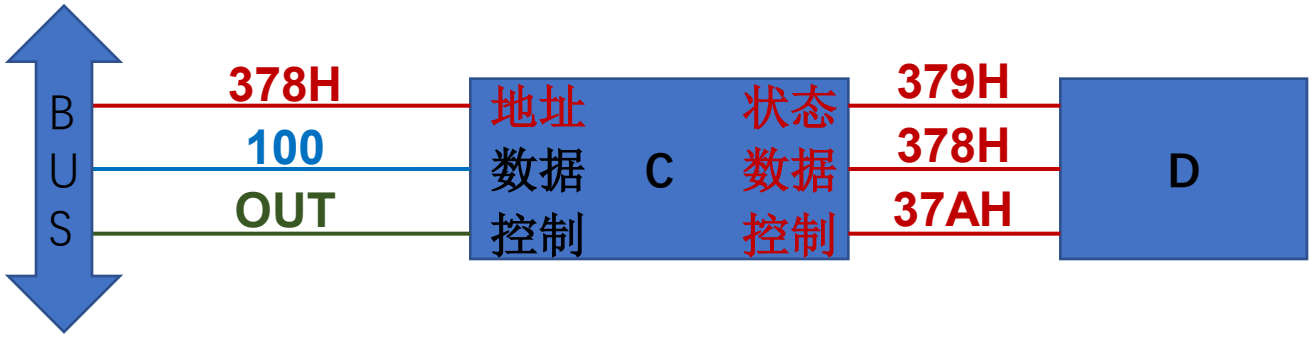
设备按共享属性分为：

- 独占设备——一次只允许一个用户/程序使用的设备（临界资源）
- 共享设备——可同时被多个用户/程序使用的设备

主机对设备的访问

设备连接到主机后将占用一组IO（端口）地址（记录于设备控制器中）。主机对设备的访问表现为使用基本的端口通讯指令（IN和OUT）与这组IO地址通讯。

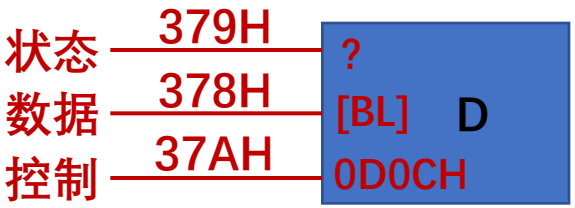
打印机接入并行端口，则占用三个地址：378H（数据信号端），37AH（控制信号端），379H（状态信号端）。



例如：向数据端口（378H）写入数据100的微观动作

主机对设备的访问

将寄存器BL（8位）中的ASCII字符输出到打印机上。




```
        MOV DX, 378H
        OUT DX, BL
        MOV DX, 379H
WAIT:   IN AL,DX
        TEST AL, 80H
        JZ WAIT
        MOV DX, 37AH
        MOV AL, 0DH
        OUT DX, AL
        MOV AL, 0CH
        OUT DX, AL
```

设备访问的软件架构

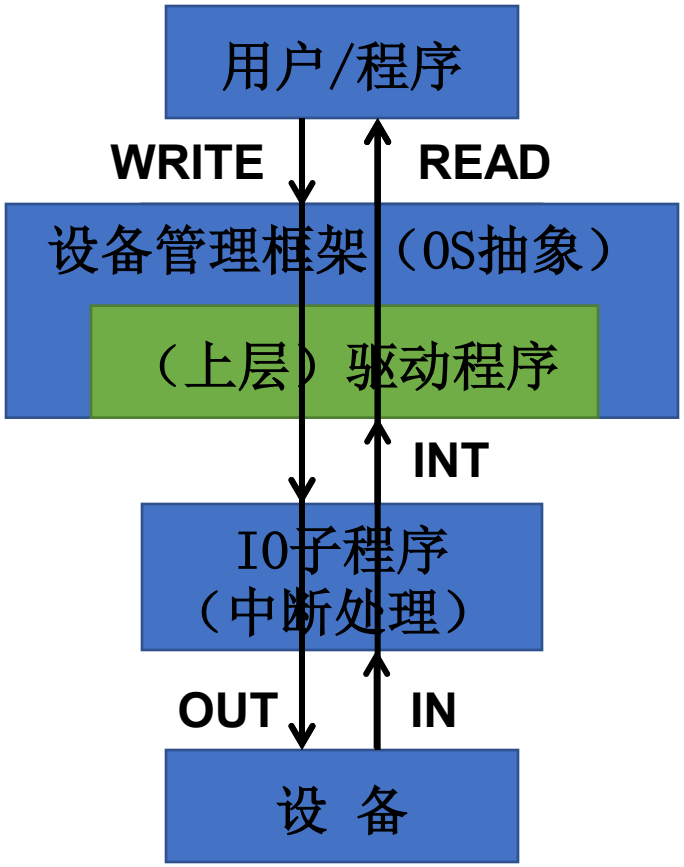
架构1. 用户/程序直接通过IN和OUT指令访问设备。

架构2. 将访问指令封装为一些子程序，用户/程序通过调用它们来访问设备。

- 用户级子程序
- 利用中断机制

架构3. 在架构2基础上，由操作系统对这些子程序实施管理、保护、配置和优化。用户/程序通过操作系统的用户接口访问设备。

在这一架构中，（上层）驱动程序利用下层IO子程序提供用户接口的具体实现。

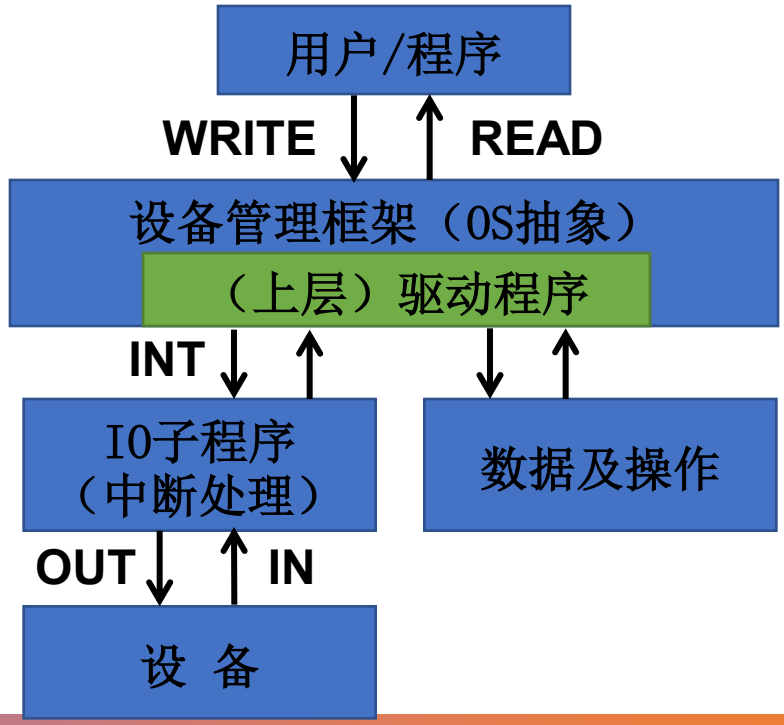


设备访问的软件架构

在这一架构中，实际由底层程序操作的设备被称作物理设备；经层层封装、管理，为用户/程序所见的设备称作逻辑设备。

对于同种逻辑设备，用户/程序按照同样的操作方式对其访问，而无需关心其对应的物理设备的特性，此种性质称作设备无关性（设备独立性）。

除物理设备外，系统还常将某些特殊数据及对其的操作进行封装和管理，使得在逻辑层面上，用户/程序可以按访问设备的方式对其进行访问。由此形成的逻辑设备称作虚拟设备。



设备的独立性

- 系统资源
- PNP技术
- IO软件的层次结构
- 设备独立性

设备管理的主要功能

- 设备的数据传输控制
- 缓冲技术
- 设备分配
- 磁盘驱动调度

数据传输控制

传输控制是指通过控制底层信号传递来实现主机与设备间数据传输以满足用户/程序的IO请求。

传输控制在其发展历史中产生四种不同的方式，方式的进化主要为了减少CPU等待时间、减轻CPU的负担，提高系统并行性。

传输控制是软硬件协作完成的，各种控制方式需要不同的硬件支持。

在软件方面，现行的软件架构中，一般由（上层）驱动程序配置和启动传输控制，由IO子程序（中断处理程序或称下层驱动程序）实现传输控制的具体操作。

数据传输控制——程序查询方式

程序查询方式是最早提出的传输控制方式。

- 进程提出IO请求并获得设备后，IO子程序将不断循环检测设备的状态，直到设备能够满足IO操作的要求（例如呈现就绪状态）时实施一个传输动作。

特点：由于不断循环检测设备状态，正在进行IO操作的进程将耗费CPU资源，无法实现处理器与设备的并行。

```
        MOV DX, 378H
        OUT DX, AL
        MOV DX, 379H
WAIT:   IN AL,DX
        TEST AL, 80H
        JZ WAIT
        MOV DX, 37AH
        MOV AL, 0DH
        OUT DX, AL
        MOV AL, 0CH
        OUT DX, AL
```

数据传输控制——中断方式

中断方式是实现系统并行处理的核心技术，利用中断来控制数据传输是现代系统的常用方式。

- 进程提出IO请求并获得设备后，若设备未就绪，则阻塞进程。
- 当设备进入就绪状态时，发出中断信号，已在系统中注册的中断处理程序唤醒进程以启动一个传输动作，而后再阻塞进程直到设备再次就绪。
- 如此循环，直到传输完成。

特点：正在等待设备就绪的进程处于阻塞状态，不消耗CPU资源，但中断发生时仍然需要CPU以实施传输动作。

数据传输控制——DMA方式

直接存储器访问（DMA）：

- 进程提出IO请求并获得设备后，将数据（或接收缓冲区）安置在内存中的位置和大小分别写入DMA控制器的基址寄存器和计数寄存器内，启动DMA过程并阻塞进程。
- DMA控制器控制指定内存区域同设备之间的数据交换。其间需要使用总线时，总线控制权将供DMA使用（硬件机制）。
- 当原进程请求的IO操作全部完成时，DMA控制器发出中断，中断处理程序唤醒进程。

特点：由DMA控制器（而非中断处理程序）来实施具体的传输动作，从而进一步减小CPU开销，但是DMA需要和处理器共享系统总线。

数据传输控制——通道方式

通道方式的特点：

- ✓ DMA方式的基础上，通道方式中使用通道处理器替代DMA控制器来实施传输动作。
- ✓ 通道处理器是一个简单的专用处理器，具有自身的指令系统，可按程序完成传输动作。这使得通道处理器较只能进行单纯传输动作的DMA控制器具有更强大的功能，例如传输纠错、数据格式转换、数据预处理等。
- ✓ 对于系统中的每一个通道，内存中有两个固定的专用存储单元分别存储通道程序的首地址（CAW通道地址字）和状态信息（CSW通道状态字）。

数据传输控制——通道方式

通道方式：

- 进程提出IO请求并获得设备后，根据所要求的IO操作，生成由通道指令组成的通道程序，并将程序首地址写入CAW中。然后启动通道并阻塞进程。
- 通道处理器从CAW中找到通道程序，并按通道程序的指令完成数据传输过程。每条指令执行之后都将通道状态写入CSW中，使得主机能够随时掌握通道运行情况。
- 若通道程序执行中出现错误，通道处理器将发出错误中断，交由中断处理程序处理错误；若通道程序顺利执行完毕也将发出完成中断，由中断处理程序唤醒原进程。

数据传输控制——通道方式

按照通道的控制方式，可分为：

- 选择通道——一台设备的传输完成后才能开始另一台设备的传输
- 多路通道——可同时进行多台设备的传输动作（微观上轮流进行）

按照通道所控制的设备特点，可分为：

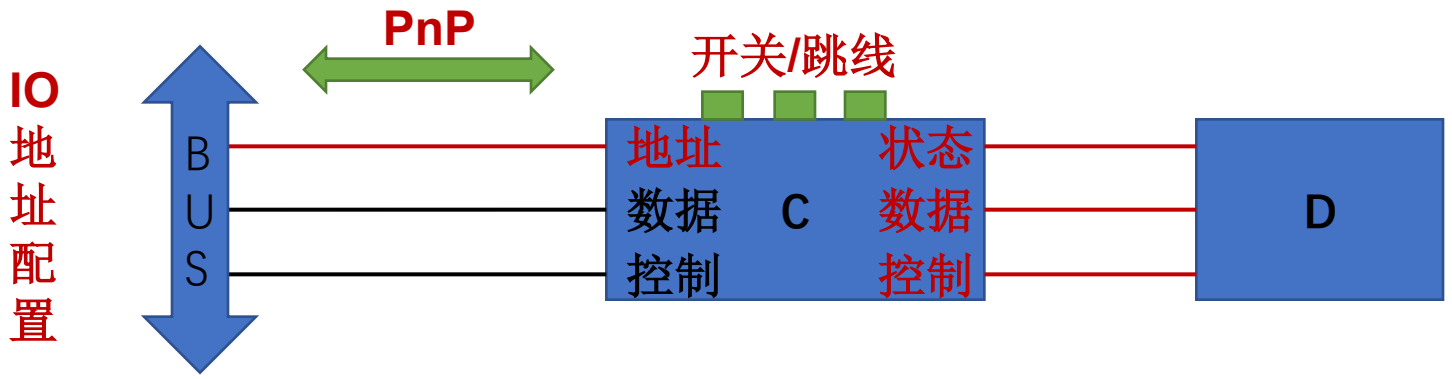
- 字节通道——控制字符设备
- 数组通道——控制块设备

字节通道通常是多路通道，而数组通道可能是选择通道或多路通道。

Unix的地址、驱动、中断管理

欲在Unix中使用一个新的设备，需要：

1. 根据实际情况对硬件参数进行配置，并将设备连接到主机上。



早期的设备通过一系列开关或跳线来手动配置硬件参数。

使用PnP技术的设备，能够在连接到主机时自动配置硬件参数，并在系统中完成各项注册任务（驱动程序安装与初始化）。

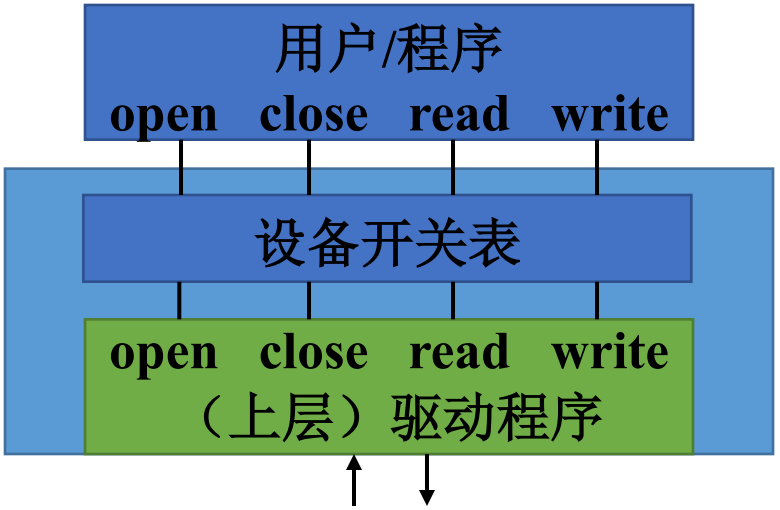
Unix的地址、驱动、中断管理

2. 向系统注册设备，以及对应的（上层）驱动程序中实现的各系统调用（open、read、write、close等）的入口（设备开关表）。

所有设备将在文件系统中以设备文件的形式呈现，

设备号	设备名	open	close	read	write
0	keyboard				
1	mouse				

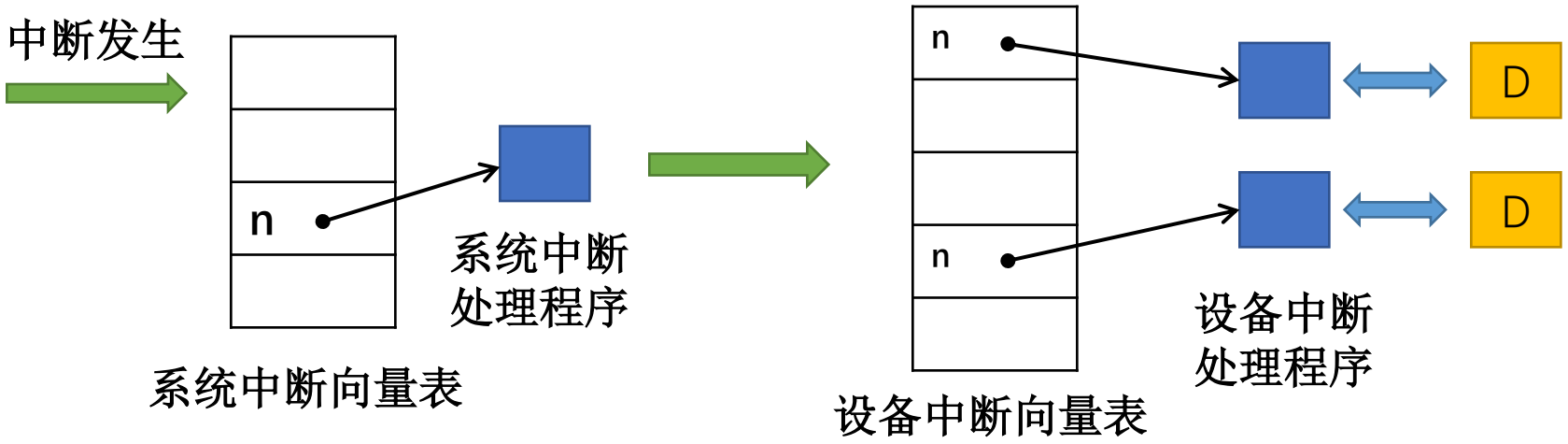
当用户/程序使用系统调用访问设备（文件）时，系统根据设备（文件）名称，在设备开关表中查找，以决定具体调用哪个（上层）驱动程序中的相应操作。



Unix的地址、驱动、中断管理

3. 向系统注册所需使用的中断号，及相应的处理程序的入口。

- 中断发生时，将跳转到系统的（而非具体设备的）中断处理程序。
- 系统的中断处理程序中依次执行注册为该中断号的所有设备中断处理程序。
- 设备中断处理程序内部需自行验证中断源和事件。



操作系统设备管理

操作系统进行设备管理的基本任务包括：

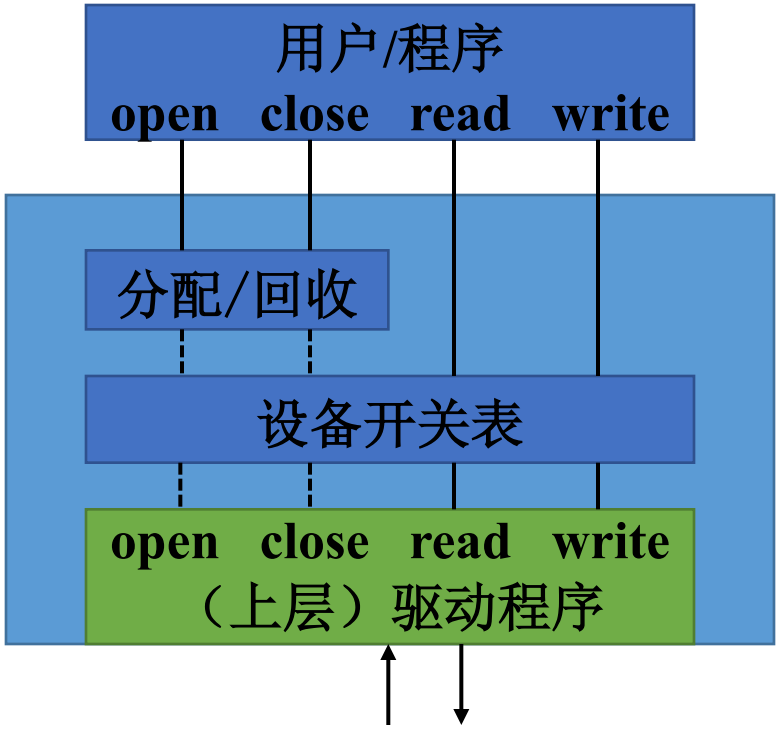
- （驱动程序和中断处理程序）实现数据传输控制
- （设备管理框架）为设备管理系统资源（IO地址、中断号 等）、驱动程序和中断处理程序
- （设备管理框架）为用户/程序分配与回收设备
- （设备管理框架）使用缓冲技术
- （设备管理框架）实现磁盘驱动调度

计算机操作系统

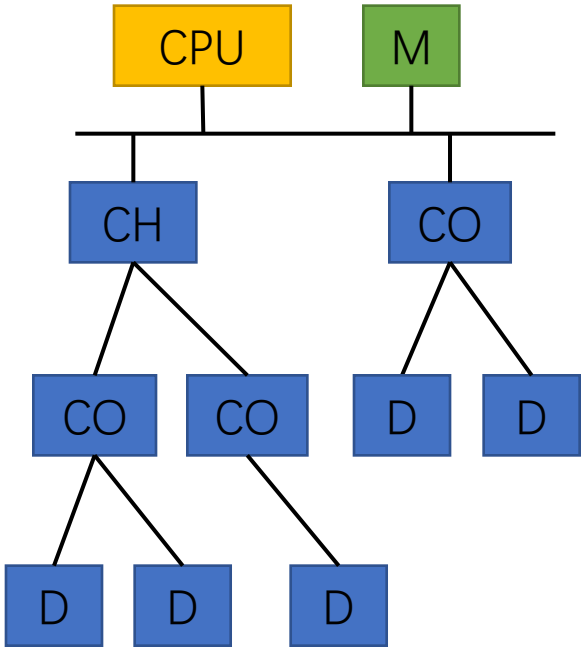
第7章 设备管理

设备的分配与回收

操作系统的IO管理框架不进行实际的设备控制和传输操作，但必须负责在进程操作设备前进行检查，避免多个进程在其open与close操作之间同时访问独占设备，即管理设备的分配（open时）和回收（close时）。



设备的分配与回收



设备的分配与存储空间的分配不同，不仅需要记录设备本身的使用情况，还需要记录设备所需线路上其他独占部件（控制器、通道等）的使用情况。

设备的分配与回收

为了记录设备及各种独占部件的使用情况，及其相互联系，必须使用四个数据结构：

- 每个通道对应一个通道控制块（CHCB）。记录通道信息，包含通道标识、状态、等待队列指针等。也称通道控制表（CHCT）。
- 每个设备控制器对应一个控制器控制块（COCB）。记录设备控制器信息，包含控制器标识、状态、等待队列指针、所连接的通道的CHCB指针等。也称控制器控制表（COCT）。

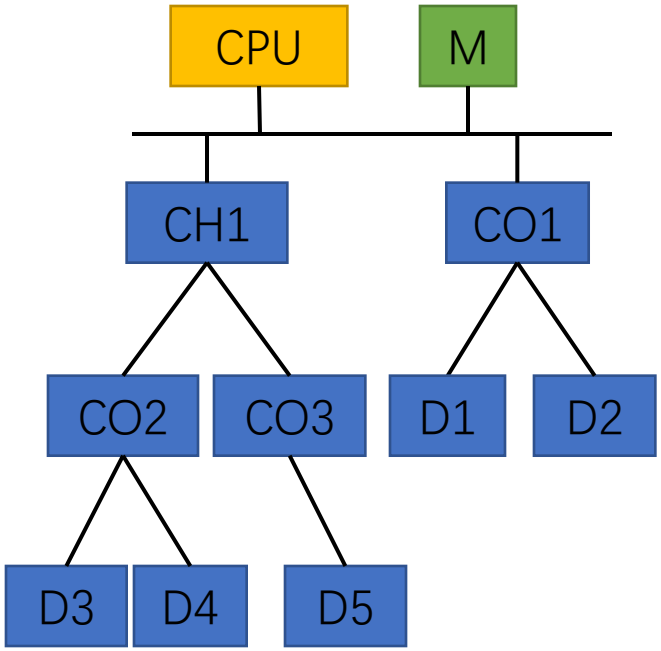
设备的分配与回收

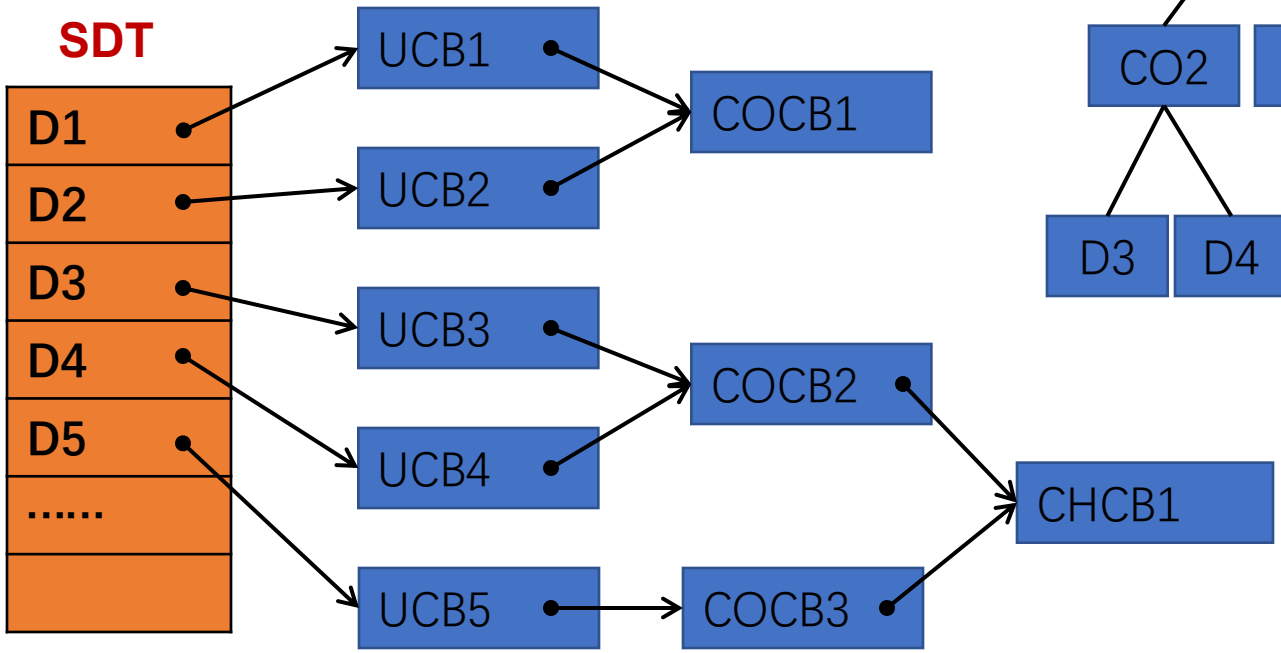
- 每个设备对应一个设备控制块（UCB或DCB）。记录设备信息，包含设备的名称、类型、标识、状态、等待队列指针、所连接到的控制器的COCB指针等。也称设备控制表（DCT）。
- 系统中有唯一的系统设备表（SDT）。用于按设备名称搜索设备并获取设备的基本信息。其中每一项记录一个设备的基本信息，包括名称、类型、标识、UCB指针、分配给的进程标识等。

设备的分配与回收

设备通道时间和设备控制器、

控制器时间和设备时间





设备的分配与回收

- ✓ 启动时，系统检测硬件连接情况，生成一个SDT及各部件的UCB、COCB或CHCB。
- ✓ 需要分配设备时，从SDT中按名找到需要的设备的UCB。若设备忙，则进程进入该UCB的等待队列。
- ✓ 否则，将设备置为忙状态（被分配），查找该设备连接的控制器的COCB。若控制器忙，则进程进入该COCB的等待队列。
- ✓ 若控制器空闲，则先将其分配后继续检查其所连接的通道的CHCB（若存在）。若通道忙，则进程进入该CHCB的等待队列。
- ✓ 若通道空闲，则将其分配。只有当设备、控制器、通道都能成功分配时，进程方可继续执行。
- ✓ 设备回收时同样依次回收设备、控制器和通道，当回收一个部件时需按一定策略唤醒该部件等待队列中的一个进程。

设备分配的安全性

操作系统在进行设备分配时需要预防资源死锁，即保证设备分配的安全性。可采用以下方法：

- 静态分配——破坏“请求保持条件”
- 单请求方式——破坏“请求保持条件”/“环路等待条件”
- 基于进程标识比较的资源抢夺或资源暂时释放：

  当进程A申请资源R，但资源R正在被进程B占用时，若A的标识较大，则采取以下二条之一：

  - 令B归还R，再处理A的申请——破坏“不剥夺条件”
  - 令A归还所有已获得的资源，然后阻塞——破坏“请求保持条件”

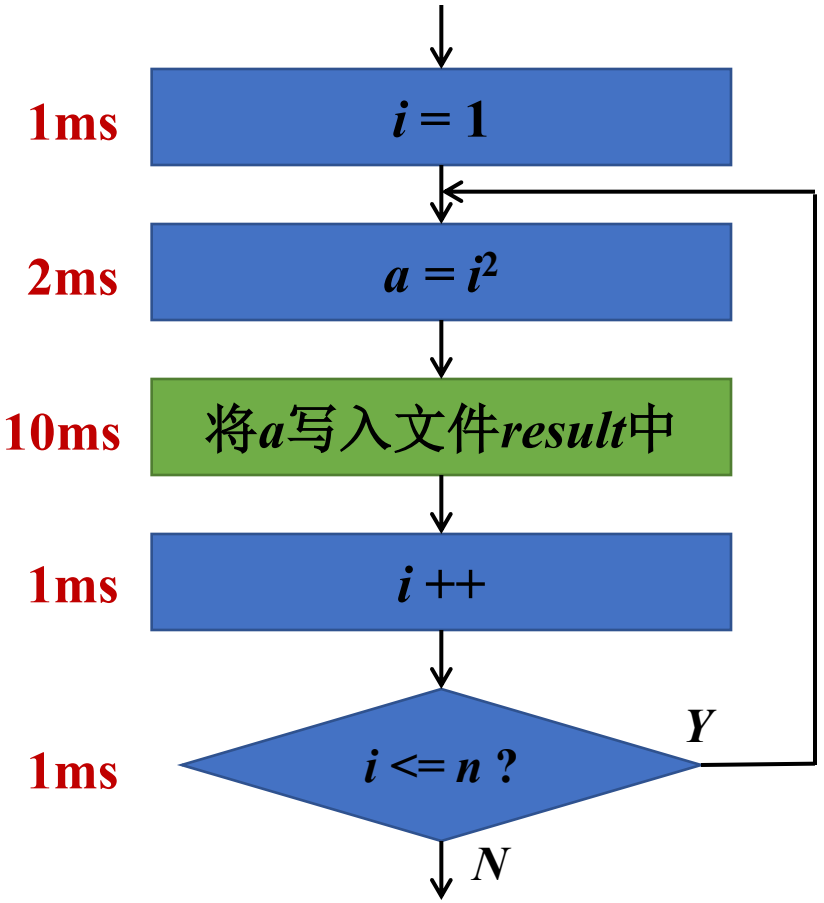
IO缓冲技术

在设备的I/O操作过程中，利用一种存储部件或其中的部分(如内存的一个区域)，暂时存放要交换的数据，将来再把数据传输到目标位置，这种数据暂存的技术称为缓冲技术。

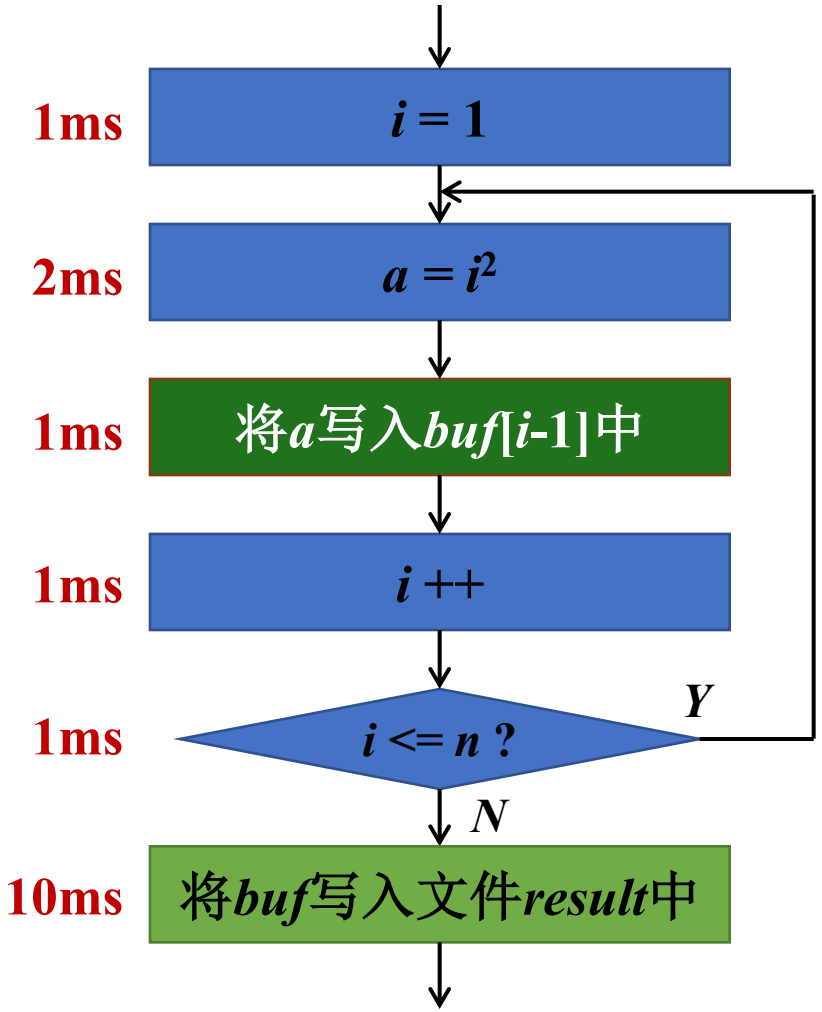
I/O缓冲的目的在于：

- 缓解设备和处理器之间的速度不匹配的矛盾，提高系统的并行程度；
- 减少I/O操作的次数；
- 减少中断次数；
- 提高系统的及时性，方便用户操作；

IO缓冲技术



耗时：14*n*+1 ms



耗时：5*n*+11 ms

IO缓冲技术

缓冲技术可分为硬件缓冲和软件缓冲，硬件缓冲技术如设备控制器中的缓冲区，软件缓冲技术其暂存部件为内存区域，根据内存区域的数量，软件缓冲分为以下三种：

- 单缓冲——系统中只存在一个缓冲区，任意时刻只能服务于一个进程对一个设备的读操作或写操作（一组IO操作）。
- 双缓冲——系统中存在两个缓冲区，可以同时服务于两组IO操作。
- 多缓冲（缓冲池）——系统中有多个缓冲区，可以同时服务于多组IO操作。

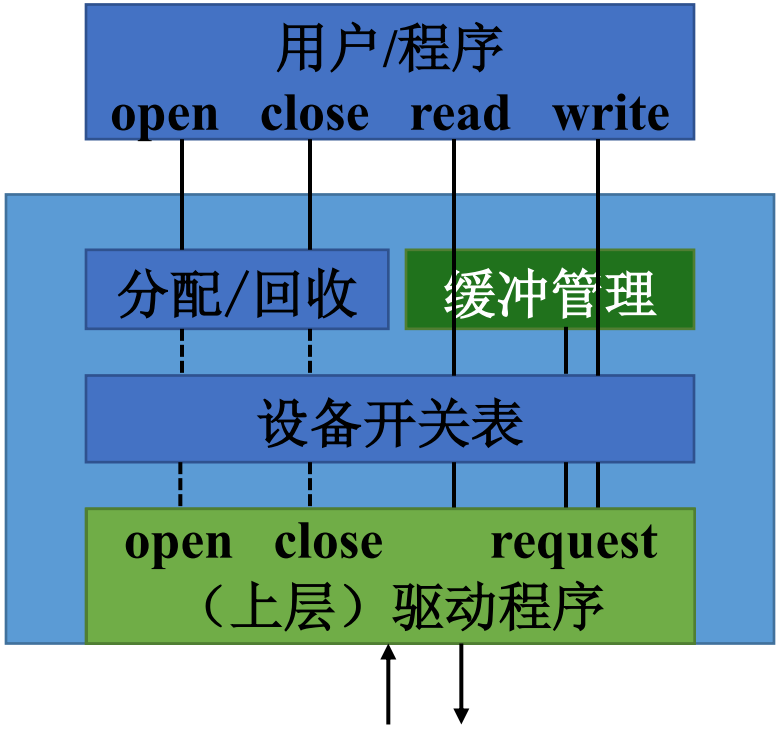
缓冲区是一种临界资源，一个缓冲区的一次使用过程只能用于一个进程对一个设备的读或写操作。设备管理框架应当实现缓冲区访问的互斥。

一个缓冲区在任意时刻可以是空缓冲区、输入缓冲区或输出缓冲区。

IO缓冲技术

在一般的设备管理软件架构中，将一个透明的缓冲管理模块添加到用户接口和上层驱动程序接口之间。

缓冲区属性指示了访问的属性（读或写），于是Unix系统中，上层驱动程序仅用一个接口request处理读写操作。



IO缓冲技术

使用缓冲技术的输出（Write）的一般过程：

1. 用户提出Write请求后，缓冲管理模块检查进程是否已取得相应的输出缓冲区，若已取得，则使用此缓冲区，否则申请一个空缓冲区，将其更改为该进程对该设备的输出缓冲区（可能阻塞原进程）。
2. 缓冲管理模块使用访存指令将数据写入缓冲区内。
3. 其间若达到一定的缓冲条件，则缓冲管理模块启动上层驱动程序，将整个缓冲区内的数据写入设备（冲洗，可能阻塞原进程），之后再继续步骤2，直到数据写入完成。

IO缓冲技术

使用缓冲技术的输入（Read）的一般过程：

1. 用户提出Read请求后，缓冲管理模块检查进程是否已取得相应的输入缓冲区，若已取得，则使用此缓冲区，否则申请一个空缓冲区，将其更改为该进程对该设备的输入缓冲区（可能阻塞原进程）。
2. 缓冲管理模块使用访存指令从缓冲区内读出数据。
3. 其间若缓冲区为空，则缓冲管理模块启动上层驱动程序，从设备读出数据到缓冲区（可能阻塞原进程），直至达到一定的缓冲条件后再继续步骤2，直到数据读出完成。

IO缓冲技术

当进程关闭（Close）设备时，将归还所有该进程针对该设备的输入/输出缓冲区（强制冲洗输出缓冲区），将他们更改为空缓冲区。

在输出时，达到缓冲条件将开始设备数据传输，而在输入时，当达到缓冲条件将停止设备数据传输。缓冲条件可以是以下之一：

- 全缓冲——设置缓冲数据量上限，当缓冲区中数据量达到上限则达到缓冲条件。
- 行缓冲——当缓冲区中出现换行符则达到缓冲条件。

计算机操作系统

第7章 设备管理

IO调度

一个实际的读写请求到达设备驱动程序时不能保证能够被立即处理。为了存放等待设备驱动程序处理的读写请求，需要维护一个请求队列。

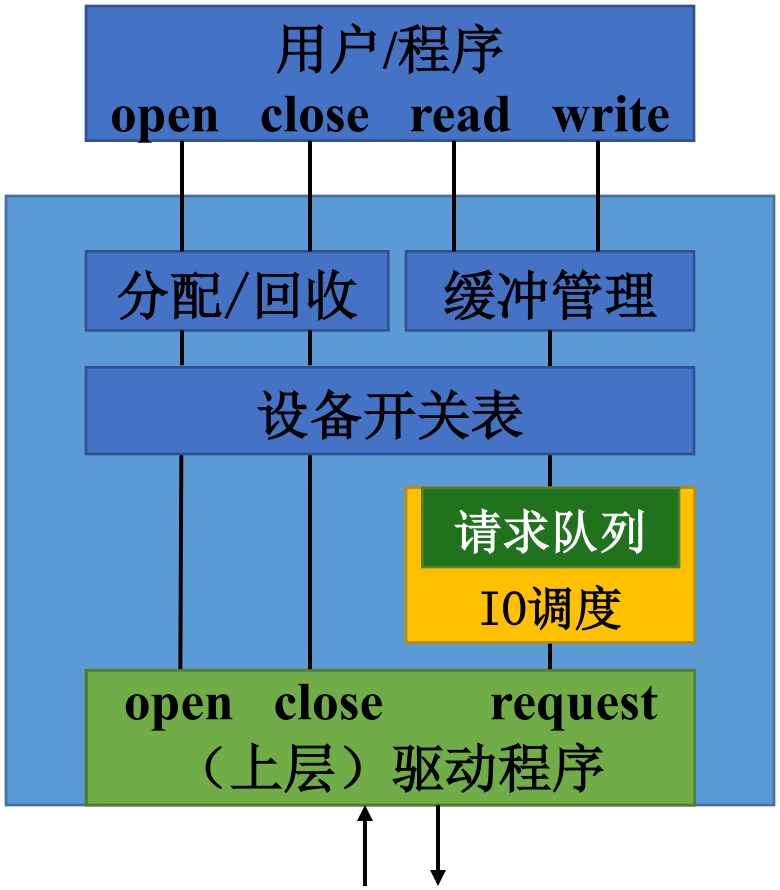
而当设备完成上一个读写请求后不但应唤醒原进程还应当将请求队列中的一个（或多个）请求送交设备驱动程序。

规划这些请求的优先关系就是IO调度。

IO调度

在Unix系统的设备管理框架中，通过IO调度器为块设备提供调度：

- 为设备注册请求队列
- 为请求队列绑定request入口
- 新请求到来时，调度器按照调度算法将其合并或插入队列中
- 设备空闲时，request通过调度器按照调度算法从队列中取出请求加以处理



磁盘的读写时间

评价IO调度算法的优劣，要根据不同设备自身的需求。

对于磁盘而言，IO调度算法的目标是缩短一组读写请求的执行时间。

磁盘设备完成一次读写动作的时间分为三部分：

1. 寻道时间——磁头从当前磁道移动到目标磁道所花费的时间

$$T_s = s_0 + s_1 + m \times n$$

s_0是磁头启动时间， s_1 是磁头稳定时间，m是跨越一个磁道的平均时间，n是跨越的磁道数。

磁盘的读写时间

2. 旋转延迟时间——磁头在磁道上搜索扇区的时间

$$T_r = r/2$$

r是磁头旋转一周的平均时间。 T_r远小于T_s。

3. 传输时间——传输数据的时间T_r

一个读写请求的执行时间： $T_a = T_s + T_r + T_t = s_0 + s_1 + m \times n + r/2 + T_t$

k个读写请求的总执行时间： $T = k(s_0 + s_1 + r/2) + \Sigma T_{ti} + m \times \Sigma n_i$

移臂调度算法

对于请求队列中的一组磁盘读写请求，运用移臂调度算法，得出执行这组请求的合理顺序，使得其间跨越磁道的总数尽量少。

常见的移臂调度算法有：

1. 先来先服务（FCFS）算法

按照请求提交的时间，依次执行。

例子 假定某磁盘的一组I/O操作的访问请求，它们的请求提出顺序依次是：55、72、100、88、93和66(I/O操作请求所在的柱面号)。当前磁盘位于90号柱面。采用移臂调度采用FCFS算法时，系统服务的顺序和磁头移动的距离以及磁头改变方向次数?

移臂调度算法

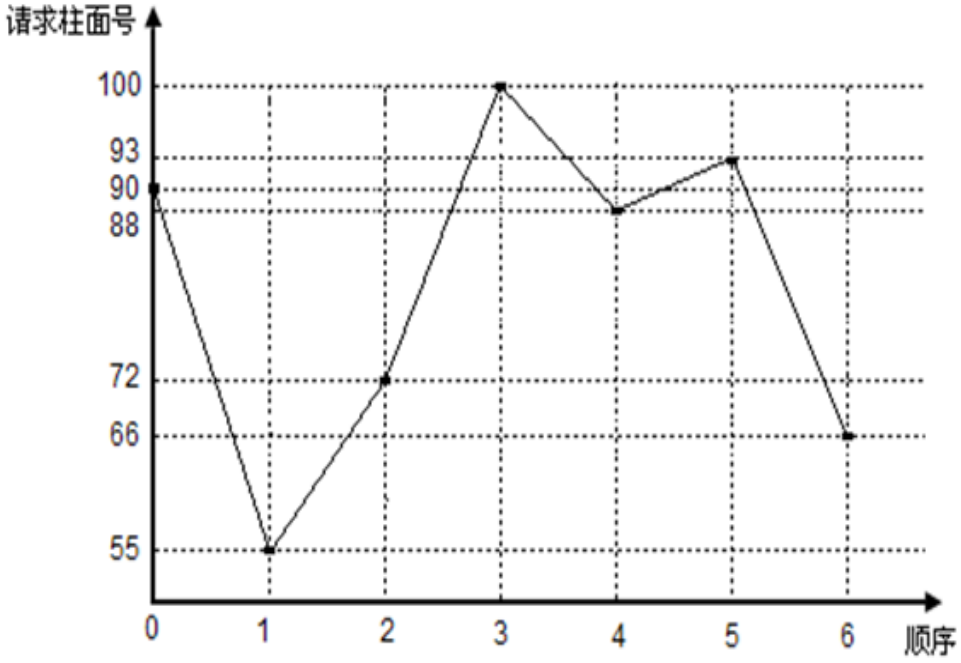
请求柱面：55、72、100、88、93、66　　当前柱面：90

服务顺序：90→55→72→100→88→93→66

移动距离：

35+17+28+12+5+27=124

磁头改变方向次数：4



移臂调度算法

2. 最短寻道优先（SSTF）算法

总是选择距离当前柱面最近的柱面

请求柱面：55、72、100、88、93、66　　当前柱面：90

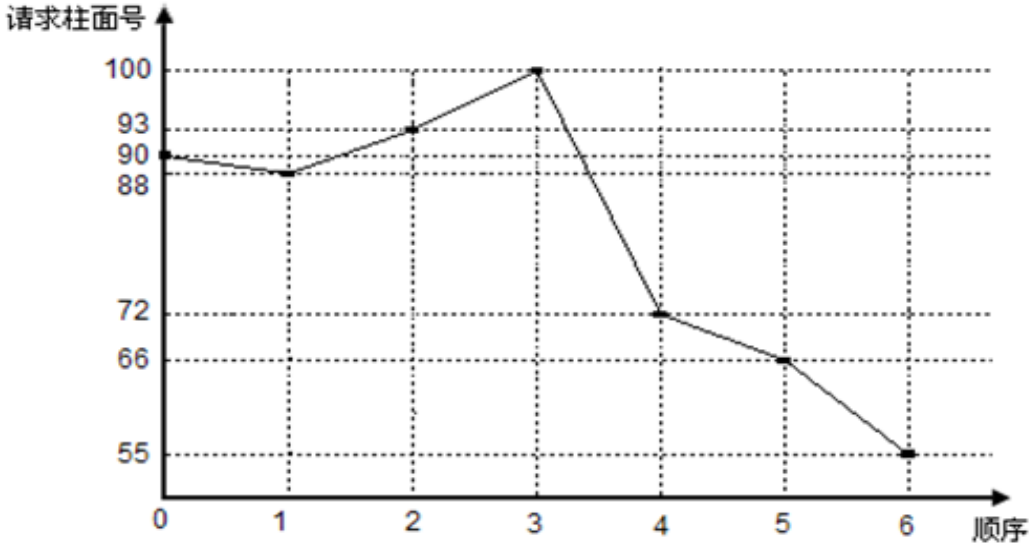
服务顺序：90→88→93→100→72→66→55

移动距离：

2+5+7+28+6+11=59

磁头改变方向次数：2

易引起“饥饿”现象



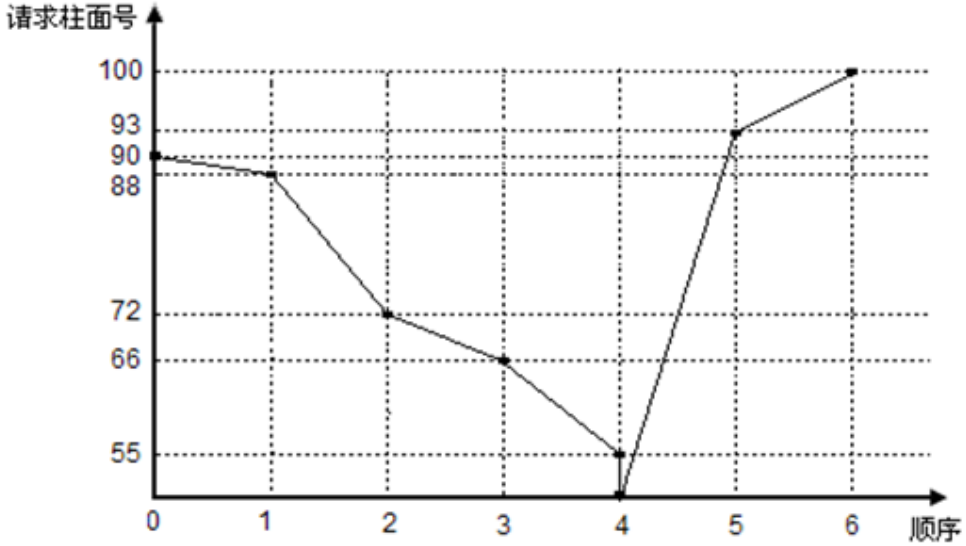
移臂调度算法

3. 扫描（SCAN）算法

总是向一个方向移动，直到磁盘边缘才改变方向

请求柱面：55、72、100、88、93、66　　当前柱面/方向：90 / 向外

服务顺序：90→88→72→66→55→0→93→100



移动距离：

2+16+6+11+55+93+7=190

磁头改变方向次数：1

磁头空移动

移臂调度算法

4. 电梯（Elevator）算法

总是向一个方向移动，直到该方向上已无请求才改变方向

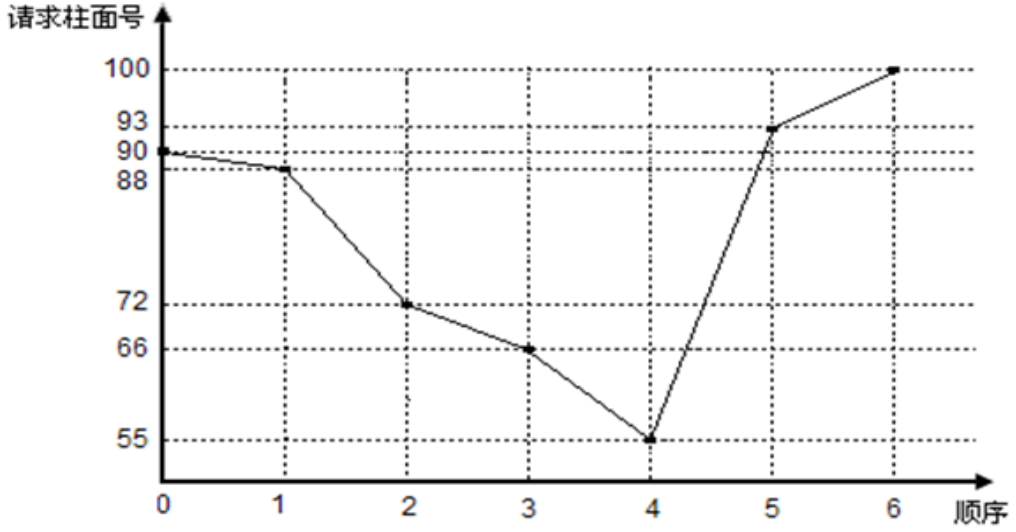
请求柱面：55、72、100、88、93、66　　当前柱面/方向：90 / 向外

服务顺序：90→88→72→66→55→93→100

移动距离：

2+16+6+11+38+7=80

磁头改变方向次数：1



移臂调度算法

移臂调度过程中，由于频繁访问临近柱面而产生的饥饿现象称为“磁臂黏着”。

N-step方法：

- 采用多个请求队列，每个请求队列最多容纳N个请求
- 当系统开始处理一个请求队列后，新的请求只能进入其他队列
- 只有当一个请求队列中的全部请求都处理完毕，系统才转向另一个请求队列

采用N-step方法后，一个当前请求队列中的读写请求最多等待N-1个其他请求后即可被处理。